

Лабораторная работа №7-8, Часть 2: Работа с SPARQL-запросами

Цель работы: Освоить язык запросов SPARQL для работы с семантическими данными. Получить практические навыки подключения к семантическому хранилищу, выполнения различных типов запросов и анализа результатов.

Стек технологий:

- **ОС:** Ubuntu 24.04 LTS
 - **Хранилище:** Apache Jena Fuseki
 - **Библиотеки:** `SPARQLWrapper`, `rdflib`, `pandas`
 - **Язык запросов:** SPARQL 1.1
 - **Форматы данных:** RDF, OWL, Turtle
-

Теоретическая часть

1. Язык SPARQL SPARQL (SPARQL Protocol and RDF Query Language) — стандартный язык запросов для RDF-данных. Основные типы запросов:

- **SELECT:** Возвращает таблицу результатов
- **CONSTRUCT:** Создает новый RDF-граф
- **ASK:** Возвращает boolean-ответ
- **DESCRIBE:** Возвращает RDF-описание ресурса

2. Apache Jena Fuseki Сервер SPARQL с веб-интерфейсом для работы с RDF-данными:

- **Поддержка SPARQL 1.1:** Полная реализация стандарта
- **Веб-интерфейс:** Интерактивное выполнение запросов
- **REST API:** Программный доступ к данным

3. Структура SPARQL-запроса

- **PREFIX:** Определение пространств имен
 - **SELECT/CONSTRUCT:** Цель запроса
 - **WHERE:** Шаблон для сопоставления
 - **FILTER:** Условия фильтрации
 - **OPTIONAL:** Необязательные совпадения
 - **ORDER BY/LIMIT:** Сортировка и ограничения
-

Задание на практическую реализацию

Этап 1: Установка и запуск Apache Jena Fuseki

1. Скачивание и установка:

```
wget https://archive.apache.org/dist/jena/binaries/apache-jena-fuseki-4.10.0.tar.gz
tar -xzf apache-jena-fuseki-4.10.0.tar.gz
cd apache-jena-fuseki-4.10.0
```

2. Запуск Fuseki сервера:

```
./fuseki-server --mem --update /ds
```

3. Проверка работы:

- Откройте браузер: <http://localhost:3030>
- Убедитесь, что сервер доступен

Этап 2: Загрузка онтологии в Fuseki

1. Загрузка данных через веб-интерфейс:

- Перейдите в [Manage Datasets](#) → [Add new dataset](#)
- Выберите [in-memory](#) хранилище
- Имя: [pizza_ds](#)
- Нажмите [Create](#)

2. Загрузка онтологии:

- Перейдите в [Upload data](#)
- Загрузите файл [pizza.owl](#)
- Нажмите [Upload](#)

3. Проверка загрузки:

- Перейдите во вкладку [Query](#)
- Выполните тестовый запрос:

```
SELECT (COUNT(*) AS ?count) WHERE { ?s ?p ?o }
```

Этап 3: Написание базовых SPARQL-запросов

1. Создание скрипта для работы с SPARQL:

```
touch sparql_queries.py
```

2. Настройка подключения:

```

from SPARQLWrapper import SPARQLWrapper, JSON, XML
import pandas as pd

# Настройка SPARQL endpoint
sparql = SPARQLWrapper("http://localhost:3030/ds/sparql")
sparql.setReturnFormat(JSON)

def run_query(query):
    sparql.setQuery(query)
    try:
        results = sparql.query().convert()
        return results
    except Exception as e:
        print(f"Ошибка выполнения запроса: {e}")
        return None

```

3. Запрос 1: Получение всех классов онтологии

```

query1 = """
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?class ?label
WHERE {
    ?class a owl:Class .
    OPTIONAL { ?class rdfs:label ?label }
}
ORDER BY ?class
"""

results1 = run_query(query1)
print("Классы онтологии:")
for result in results1["results"]["bindings"]:
    print(f"{result['class']['value']} - {result.get('label', {})}.get('value', 'No label')}")

```

4. Запрос 2: Поиск всех пицц

```

query2 = """
PREFIX pizza: <http://www.co-ode.org/ontologies/pizza/pizza.owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pizza ?name
WHERE {
    ?pizza a pizza:Pizza .
    ?pizza rdfs:label ?name .
}
ORDER BY ?name
"""

```

```

results2 = run_query(query2)
print("\nВсе пиццы:")
for result in results2["results"]["bindings"]:
    print(result['name']['value'])

```

Этап 4: Сложные запросы с фильтрацией

1. Запрос 3: Пиццы с определенной начинкой

```

query3 = """
PREFIX pizza: <http://www.co-ode.org/ontologies/pizza/pizza.owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pizza ?name ?topping
WHERE {
    ?pizza a pizza:Pizza .
    ?pizza rdfs:label ?name .
    ?pizza pizza:hasTopping ?toppingObj .
    ?toppingObj rdfs:label ?topping .
    FILTER (CONTAINS(LCASE(?topping), "mushroom"))
}
"""

results3 = run_query(query3)
print("\nПиццы с грибами:")
for result in results3["results"]["bindings"]:
    print(f"{result['name']['value']} - {result['topping']['value']}")

```

2. Запрос 4: Статистика по начинкам

```

query4 = """
PREFIX pizza: <http://www.co-ode.org/ontologies/pizza/pizza.owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?topping (COUNT(?pizza) AS ?count)
WHERE {
    ?pizza a pizza:Pizza .
    ?pizza pizza:hasTopping ?toppingObj .
    ?toppingObj rdfs:label ?topping .
}
GROUP BY ?topping
ORDER BY DESC(?count)
LIMIT 10
"""

results4 = run_query(query4)
print("\nПопулярные начинки:")

```

```
for result in results4["results"]["bindings"]:
    print(f"{result['topping']['value']}: {result['count']['value']}")
```

Этап 5: CONSTRUCT-запросы для создания новых данных

1. Запрос 5: Создание RDF-графа вегетарианских пицц

```
query5 = """
PREFIX pizza: <http://www.co-ode.org/ontologies/pizza/pizza.owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <http://example.org/vegetarian#>

CONSTRUCT {
    ?pizza ex:isVegetarian true .
    ?pizza ex:hasTopping ?topping .
}
WHERE {
    ?pizza a pizza:Pizza .
    ?pizza rdfs:label ?name .
    ?pizza pizza:hasTopping ?toppingObj .
    ?toppingObj rdfs:label ?topping .
    FILTER NOT EXISTS {
        ?toppingObj a pizza:MeatTopping .
    }
}
"""

# Для CONSTRUCT запросов меняем формат вывода
sparql.setReturnFormat(XML)
results5 = run_query(query5)
print("CONSTRUCT запрос выполнен")

# Сохранение результатов
with open("vegetarian_pizzas.rdf", "w") as f:
    f.write(results5.toxml())
```

Этап 6: Работа с онтологией через RDFLib

1. Альтернативный способ работы с данными:

```
from rdflib import Graph, Namespace
from rdflib.plugins.stores import sparqlstore

# Создание графа с SPARQL endpoint
store = sparqlstore.SPARQLUpdateStore()
store.open(('http://localhost:3030/ds/sparql',
           'http://localhost:3030/ds/update'))

g = Graph(store)
```

```

# Определение namespace
PIZZA = Namespace("http://www.co-ode.org/ontologies/pizza/pizza.owl#")
RDFS = Namespace("http://www.w3.org/2000/01/rdf-schema#")

# Запрос через RDFLib
query6 = """
SELECT ?pizza ?name
WHERE {
    ?pizza a pizza:Pizza .
    ?pizza rdfs:label ?name .
}
LIMIT 5
"""

results6 = g.query(query6, initNs={"pizza": PIZZA, "rdfs": RDFS})
print("\nРезультаты через RDFLib:")
for row in results6:
    print(f"{row.pizza} - {row.name}")

```

Этап 7: Создание комплексных отчетов

1. Генерация отчета по онтологии:

```

def generate_ontology_report():
    queries = {
        "total_classes": """
            SELECT (COUNT(DISTINCT ?class) AS ?count)
            WHERE { ?class a owl:Class }
        """,
        "total_properties": """
            SELECT (COUNT(DISTINCT ?prop) AS ?count)
            WHERE { ?prop a owl:ObjectProperty }
        """,
        "total_individuals": """
            SELECT (COUNT(DISTINCT ?ind) AS ?count)
            WHERE { ?ind a owl:NamedIndividual }
        """
    }

    report = {}
    for name, query in queries.items():
        results = run_query(query)
        if results:
            count = results["results"]["bindings"][0]["count"]["value"]
            report[name] = count

    # Сохранение отчета
    df = pd.DataFrame([report])
    df.to_csv("ontology_report.csv", index=False)
    return report

```

```
ontology_stats = generate_ontology_report()
print("\nСтатистика онтологии:")
for key, value in ontology_stats.items():
    print(f"{key}: {value}")
```

Этап 8: Интеграционные тесты

1. Тестирование различных endpoint:

```
def test_endpoints():
    endpoints = [
        "http://localhost:3030/ds/sparql",
        "http://dbpedia.org/sparql",
        "http://query.wikidata.org/sparql"
    ]

    test_query = "SELECT (COUNT(*) AS ?count) WHERE { ?s ?p ?o } LIMIT 1"

    for endpoint in endpoints:
        try:
            sparql = SPARQLWrapper(endpoint)
            sparql.setQuery(test_query)
            sparql.setReturnFormat(JSON)
            results = sparql.query().convert()
            print(f"{endpoint}: Работает ({results['results']['bindings'][0]
['count']['value']} triplets)")
        except:
            print(f"{endpoint}: Не доступен")

test_endpoints()
```

Требования к оформлению и отчету

Критерии оценки для Части 2:

- **Удовлетворительно:** Успешно выполнены Этапы 1-3 (запуск Fuseki, базовые SELECT-запросы). Получены списки классов и пицц.
 - **Хорошо:** Дополнительно успешно выполнен Этап 4-5 (сложные запросы с фильтрацией, CONSTRUCT-запросы). Создан RDF-граф вегетарианских пицц.
 - **Отлично:** Все задания выполнены в полном объеме. Реализованы Этапы 6-8: работа через RDFLib, генерация отчетов, интеграционные тесты. Проанализирована структура онтологии через SPARQL.
-

Рекомендуемая литература

1. **SPARQL 1.1 Specification:** <https://www.w3.org/TR/sparql11-query/>

2. **Apache Jena Documentation:** <https://jena.apache.org/documentation/fuseki2/>
3. **SPARQLWrapper Documentation:** <https://sparqlwrapper.readthedocs.io/>
4. **RDFLib Documentation:** <https://rdflib.readthedocs.io/>
5. **Learning SPARQL:** <https://www.learnsparql.com/>